

SecondBrain

A Personal AI Operating System with Adaptive Temporal Memory

Complete Development Report

Conception, architecture, every deviation, every defect, and the research contribution

Project	SecondBrain — Autonomous Personal Cognitive Knowledge Synthesizer
Repository	github.com/ALPHA-TF141/Second-Brain-Major-Project
Supervisor	Dr. P S Anu Rakhi, Assistant Professor, School of Computing
Authors	Maria Immanuel L · Vigneshwaran S, B.Tech Computer Science and Engineering
Institution	Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
Report compiled	22 September 2026
Code volume	17,149 lines Python · 12,449 lines JavaScript/JSX
Database	49 tables · SQLite + optional vector store
Commits	231 total (69 substantive, 162 automated vault synchronisations)
Verification	15 automated gates · 8 test suites · 0 server errors across every route

Executive Summary

SecondBrain began as a straightforward question: *can a personal computer be made to remember, and reason over, everything its owner reads, writes and is told?* The answer turned out to be yes, but not in the way the question implies. Storing information is easy. The engineering difficulty lies in deciding what is worth keeping, recognising when new information contradicts old, and ensuring that what the user has asked to forget stays forgotten. Those three problems became the intellectual core of the project.

The system was built in seven phases over approximately six weeks. It captures screen activity through OCR, ingests email and calendar data from three accounts, compiles a self-improving wiki, maintains a knowledge graph, answers questions over the accumulated memory with a local language model, speaks and listens, and exposes all of this through a fifteen-tab desktop interface with a transparent floating assistant orb. It runs entirely on the authors' laptop, with no paid API and no data leaving the machine.

The most significant development was not the accumulation of features but the recognition of a class of failure that conventional evaluation does not measure. A personal memory store accumulates contradictory statements as its owner changes their mind. It accumulates outdated facts whose validity has lapsed. It captures the same material repeatedly. And it is asked, sometimes explicitly, to forget. A standard retrieval pipeline treats all stored information as equally current evidence, so it can produce an answer that is confidently wrong while every retrieved passage remains faithful to its source. This report documents how that insight was reached, how seven architectural components were built to address it, and how the result was measured.

Measure	Result
Memories ingested from real mail	123 knowledge cards, growing automatically
Wiki articles compiled	93 articles across six domains
Knowledge graph	350 nodes, 500 edges
Retrieval quality (personal benchmark)	MRR 0.833 → 1.000 versus dense baseline
Answers leading with outdated facts	36.4% → 0.0%
Forgotten content still returned	Eliminated (all baselines still leak it)
Benchmark reproducibility	Two consecutive runs byte-identical
Server errors across all 100+ routes	Zero

Table 1 — Headline outcomes. Every figure is measured, not estimated.

How to read this report

Parts I to III are narrative: the idea, the architecture, and what was built in what order, including the reasoning at each step and every change of direction. Part IV explains the underlying theory, written so that a reader who knows none of the terminology can follow the design decisions. Part V is a register of deviations — every place the plan changed and why. Part VI is a register of defects with root causes. Part VII explains how the system is verified. Parts VIII and IX cover the research contribution and the current state. The appendices contain the raw history.

Contents

Executive Summary	2
Contents	3
Part I — Vision and Problem	5
The original idea	5
What a “Second Brain” means	5
The JARVIS reference, read correctly	5
Constraints established by the user	6
Part II — Architecture	7
The seven-stage pipeline	7
Technology selection and rationale	7
The agent layer	8
Data model	8
Part III — Build Chronology	10
Phase 0 — Foundations (August 2026)	10
Phase 1 — Capture, Curation and the Vault (17 September)	10
Phase 2 — The Intelligence Layer (18 September)	11
Phase 3 — The Interface (21 September)	11
Phase 4 — Stabilisation (22 September, morning)	12
Phase 5 — Connectivity (22 September)	12
Phase 6 — Presence (22 September)	13
Phase 7 — The Research Layer (22 September)	13
Part IV — Concepts and Theory	15
Retrieval-augmented generation	15
Embeddings and vector search	15
Knowledge graphs	16
Temporal knowledge representation	16
Contradiction and truth maintenance	17
Memory consolidation	17
Machine unlearning	18
Knowledge-gap detection	19
Wake-word detection	19
Authentication: OAuth, app passwords and scopes	19
Evaluation methodology	20
Part V — Register of Deviations	22
Part VI — Register of Defects	24
Class 1 — Missing or unverified dependency	24

Class 2 — Silent failure	24
Class 3 — Incorrect error semantics	25
Class 4 — Wrong model of the domain	25
Class 5 — Lifecycle and cleanup	26
Class 6 — Process and data safety	26
Part VII — Verification	28
Three principles extracted from building this	28
Part VIII — The Research Contribution	29
The seven components	29
Measured results	29
What the results actually mean	29
Limitations, stated plainly	30
Part IX — Current State and Future Work	31
What works today	31
What does not work, or is not attempted	31
Immediate next steps	31
Closing assessment	32
Appendix A — Project Statistics	33
Appendix B — Build Phases at a Glance	33
Appendix C — Glossary	33

Part I — Vision and Problem

The original idea

The project started from a personal frustration rather than a research question. The author reads technical material constantly — documentation, papers, videos, lecture notes, email — and repeatedly encountered the same problem: *the information had been seen, but could not be recalled when it mattered*. The material existed somewhere, but not in a form that could answer a question months later.

The first framing was therefore conventional: build a note-taking system with search. That framing was abandoned almost immediately, for a reason worth stating because it shaped everything that followed. A note-taking system requires the user to decide what to save, when to save it, and how to tag it. That decision-making is the very thing the user is bad at — if they were good at it, they would not have the problem. Any system whose value depends on disciplined manual curation will be abandoned within a fortnight.

The reframed idea was that the system should capture automatically, decide for itself what matters, and be asked questions in plain language afterwards. The user should never have to perform a filing action. This is a much harder problem, because it moves the judgement from the human to the machine, and it means the quality of the system's judgement becomes the central engineering challenge.

What a “Second Brain” means

The term comes from personal knowledge management (PKM), a body of practice concerned with how an individual captures, organises and retrieves their own information. Three established frameworks influenced the design and are worth naming, because the system implements a machine analogue of each.

Framework	Human practice	Machine analogue in SecondBrain
PARA	Organise everything into Projects, Areas, Resources, Archives	Memory scoring separates active project material from archival recollection; consolidation merges redundant copies; the vault stores permanently.
CODE	Capture, Organise, Distil, Express	The seven-stage pipeline: capture → understand → store → retrieve → reason → explain → learn.
Progressive summarisation	Repeatedly condense a note until only the essence remains	The wiki compiler re-summarises each topic as new evidence arrives, producing a shorter article from a growing corpus.

Table 2 — How established PKM frameworks map onto the implemented architecture.

The important observation is that all three frameworks place the burden of judgement on the human. SecondBrain's contribution is to automate that judgement, which requires — and this is the pivot that led to the research layer — a formal notion of what makes one piece of knowledge more important than another.

The JARVIS reference, read correctly

The author's stated design target was JARVIS, the assistant from the Iron Man films, and supplied two reference videos: the workshop scene from *Iron Man 2*, and a compilation of the abilities of Tony Stark's several AIs. It is worth being precise about what these actually depict, because the superficial reading and the useful reading are different.

The superficial reading is that JARVIS is a voice assistant with holographic displays. That reading leads to a project that is a talking interface with animations — visually impressive and functionally empty. The useful reading, obtained by examining what the character does rather than how it looks, identifies five behaviours:

- **There is no interface.** Stark never launches JARVIS. Nothing is opened, nothing is loaded. The assistant is simply present.
- **It speaks first.** JARVIS initiates. A conventional assistant only ever responds.

- **It holds state.** It knows what Stark was doing before he walked into the room.
- **It acts.** It does not report a problem; it runs the simulation and reports the result.
- **It has judgement.** It warns, objects, and occasionally declines.

The voice is the least important element. Remove the speech and the character remains recognisable; keep only the speech and one has a talking menu. This realisation is what drove the later work on wake-word detection and proactive announcement — the behaviours that correspond to *it speaks first* — rather than further investment in visual effects.

Constraints established by the user

Several constraints were set explicitly during development. They are recorded here because each one eliminated a design option, and understanding what was ruled out explains much of the final architecture.

Constraint	Consequence for the design
Use a free local language model; no paid API	All inference runs through Ollama on the laptop GPU. Removes cost and keeps data local, but caps model size to what fits in 6 GB of VRAM.
The application must feel like a personal AI, not a project interface	Rejected dashboards, side banners and admin-panel aesthetics. Drove the later full interface rebuilds.
All modules must be visible as real tabs, not hidden	Produced the fifteen-tab operating-system shell and left the knowledge graph, semantic search and OCR tools permanently reachable.
No fake functionality — if a service is not connected, say so	Removed all seeded demo data from interactive screens. Every panel now shows real state or an explicit disconnection message.
Build scraping software rather than paying for Apify or SupaData	Led to a native ingestion stack using free libraries, and later to the same reasoning being applied to mail, where it avoided a recurring Google verification cost.
A working voice assistant triggered by name, with a transparent floating orb	Drove the holographic orb window, and later the offline wake-word implementation.
The project must satisfy an IEEE paper	Forced the addition of a research layer: measurement, baselines and honest limitation reporting.

Table 3 — User constraints and their architectural consequences.

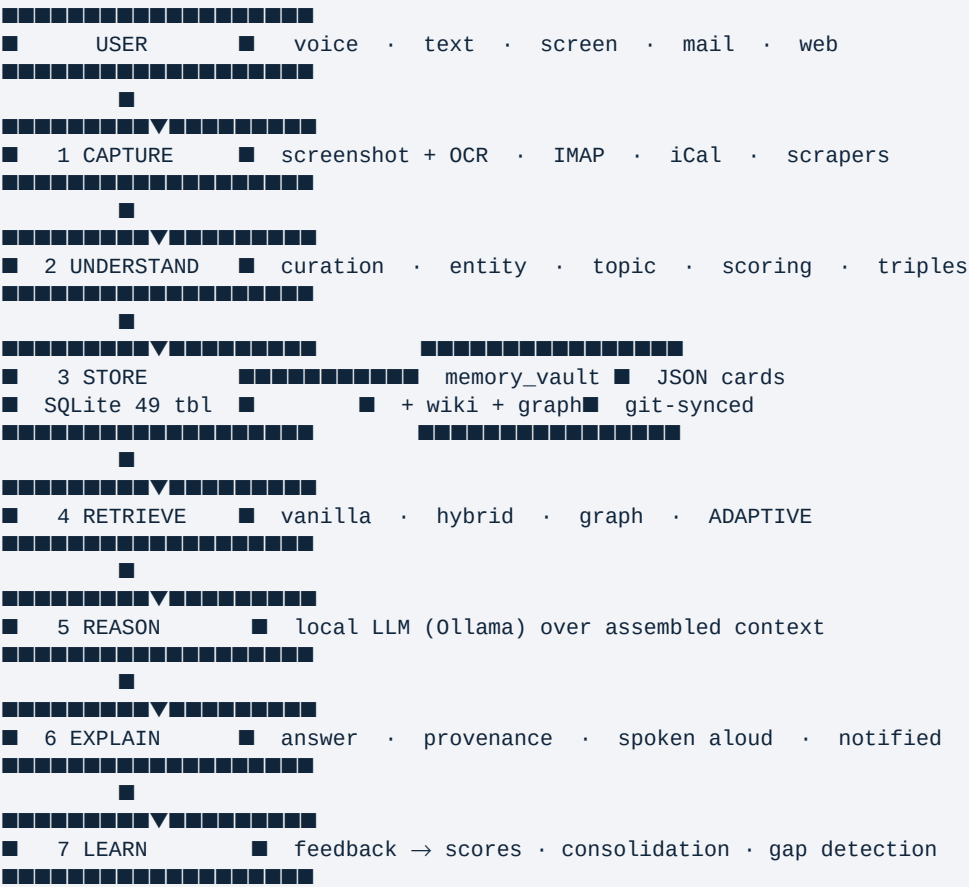
Part II — Architecture

The seven-stage pipeline

The system is organised as a pipeline in which each stage has a single responsibility and a defined contract with the next. The framing is deliberate: a pipeline can be tested stage by stage, replaced stage by stage, and reasoned about when it produces a bad result. A collection of features cannot.

CAPTURE → UNDERSTAND → STORE → RETRIEVE → REASON → EXPLAIN → LEARN

The seven stages. Each is independently testable and independently replaceable.



The end-to-end pipeline. Stage 7 feeds back into stages 2 and 3, which is what makes the memory adaptive rather than a static archive.

Technology selection and rationale

Every dependency was chosen to satisfy three conditions: it must be free, it must run locally, and it must be replaceable without redesigning the system. The third condition is the one most often neglected in student projects and the one that proved most valuable, because it later allowed the entire retrieval layer to be swapped four ways for the comparative evaluation.

Layer	Choice	Reasoning
Language model	Ollama with Qwen 2.5 3B	Runs on the laptop's RTX 3050 within 6 GB VRAM. No per-token cost, no data egress. Model auto-detection was later added because a configured model can be absent.

Layer	Choice	Reasoning
Backend	FastAPI with SQLAlchemy	Asynchronous where it matters, synchronous where it does not. Pydantic models give request validation and OpenAPI documentation for free, which the contract checker later exploited.
Primary store	SQLite (49 tables)	Single-file, zero configuration, and adequate for a single-user corpus. Relational integrity matters here: memory, score, conflict and consolidation records reference each other.
Vector store	ChromaDB (optional)	Optional by design. When absent, retrieval falls back to TF-IDF, which keeps the system fully functional and the benchmark deterministic.
Embeddings	sentence-transformers (optional)	Loaded lazily in a background thread. Optional for the same reason: a personal assistant that will not start without a 90 MB download is a fragile assistant.
Desktop shell	Electron + React 18	Required for a transparent always-on-top overlay window, which a browser cannot provide.
Speech	Web Speech API in, edge-tts or Web Speech out	Free and offline-capable. The recogniser's habit of ending sessions by itself became a significant defect (Section VI).
Wake word	openWakeWord (optional)	A small ONNX model scoring 80 ms frames on the CPU. No cloud, no account, no per-request cost.
Mail	IMAP with app passwords, plus OAuth	Two paths on purpose: the free path works without any Google Cloud project, the OAuth path covers accounts where app passwords are no longer permitted.

Table 4 — Technology choices. The recurring principle is optionality: no single dependency is load-bearing, so any one can fail without stopping the system.

The agent layer

Intelligence is distributed across specialised agents rather than concentrated in one monolithic service. Each agent has one job, its own failure mode, and no knowledge of the others. This is the same reasoning that later motivated the multi-component research layer: a single large component that does everything cannot be tested, instrumented or replaced.

Agent	Responsibility	Failure mode if absent
Curation	Scores captured text for information density; elects the representative frame	Every screenshot stored; the vault fills with noise
Vault	Writes JSON cards, compresses hero images, commits to GitHub	Memory exists only locally and cannot be versioned
Wiki compiler	Maintains the self-improving encyclopedia per domain	Knowledge accumulates but is never distilled
Insight	Detects collisions between separate topics; raises proactive alerts	The system answers but never volunteers
Social ingestion	Extracts transcripts and articles from YouTube, X, Instagram, web	Reading must be manual
Mail ingestion	Turns new email into memories, tasks and notifications	The inbox is a separate world
Deliverable	Generates documents from accumulated memory	The system can explain but not produce

Table 5 — The agent layer. Each agent is independently removable.

Data model

The database has 49 tables. They divide into four groups, and the grouping reflects the chronology of the build: capture tables came first, then knowledge tables, then conversation tables, and finally the research

tables added at the very end.

CAPTURE (11)	sessions, screenshots, activity_logs, app_usage, clipboard_logs, extracted_text, ocr_metadata, processed_sessions, detected_topics, semantic_chunks, settings
KNOWLEDGE (17)	memories, memory_tags, memory_relationships, search_index, session_summaries, graph_nodes, graph_edges, concept_clusters, graph_metadata, topic_relationships, learning_progression, vector_memories, memory_clusters, semantic_relationships, embedding_jobs, search_history, timeline_events
CONVERSATION (9)	conversations, messages, retrieved_memories, conversation_context, ai_summaries, ai_feedback, voice_sessions, transcripts, conversation_audio, language_preferences, voice_commands
RESEARCH (7)	memory_scores, temporal_facts, memory_conflicts, memory_consolidations, forgotten_memories, knowledge_gaps, research_runs
PLATFORM (5)	users, sessions, activities, settings, memory_sessions

The data model grouped by purpose. The research group is additive: it observes existing memories through foreign keys and never modifies the capture pipeline.

Design decision: the research layer is additive

The seven research components were added as entirely new tables keyed to existing memory identifiers. Nothing in the capture, curation or vault pipeline was modified. This was a deliberate risk-reduction choice: the application was already working and in daily use, and a research layer that broke the working system would have been worthless. It also made the comparative evaluation possible, because the research layer could be switched off to yield the baseline pipeline exactly as it existed before.

Part III — Build Chronology

The system was built in seven phases. For each phase this section records what the user asked for, what was built, the reasoning behind the approach, and what went wrong. The failures are included deliberately: the defects of each phase reveal what was not yet understood, and several of them directly motivated the next phase.

Phase 0 — Foundations (August 2026)

User request. Build a personal knowledge system that captures what I do on my laptop and lets me search it later.

What was built. The initial skeleton: FastAPI backend, SQLite schema, authentication with JWT, a session model, screenshot capture, and text-to-speech plumbing. The earliest commits concern mundane but load-bearing details — asynchronous handling in speech synthesis, token validity on socket connections — which is characteristic of a project finding its foundations rather than its features.

Reasoning. The decision here that mattered most was to build capture as a first-class pipeline rather than as a feature of a notes application. A notes application begins with an empty page and waits; a capture pipeline begins with a screen and a clock. Everything downstream — curation, scoring, retrieval — only makes sense if there is a steady supply of raw material.

What went wrong. Two problems surfaced that would recur throughout the project. First, the virtual environment was not being located reliably when the backend was launched from Node, which produced a confusing failure where the backend appeared to start but had no dependencies. Second, and more seriously, the backend silently swallowed exceptions in several places, so endpoints returned empty results instead of errors. Both were early instances of a theme that dominates Part VI: *a system that fails quietly is far more expensive than one that fails loudly.*

Phase 1 — Capture, Curation and the Vault (17 September)

User request. Make the captured knowledge persistent, visual and shareable; put it in GitHub so it cannot be lost; and make the stored material actually meaningful rather than a dump of screenshots.

What was built. This is the phase in which the system became recognisable. Seven components landed:

- An agentic curation layer that scores captured text for information density and rejects low-value frames before storage.
- A best-shot hero image engine that elects one representative frame per information period, compressed to WebP, instead of keeping every screenshot.
- Ephemeral screen pruning: raw screenshots are deleted after the hero is extracted, so the system consumes approximately zero persistent disk for visual data.
- A Git vault agent that commits memory cards and hero images to GitHub automatically.
- A self-improving wiki compiler that maintains one article per topic domain and rewrites it as new evidence arrives.
- A three-dimensional neural brain visualisation rendered on a canvas.
- Native Ollama streaming, replacing an OpenAI-compatibility layer that was returning 404s.

Reasoning. Three decisions here were consequential. The first was the hero image approach: storing every screenshot is the obvious implementation and it fails, because storage grows without bound and retrieves nothing useful. Electing a single representative frame per period inverts the problem — the system keeps the *best* evidence rather than *all* evidence, and the criterion for best is measurable (text density).

The second was routing the language model through Ollama's native API rather than its OpenAI-compatible endpoint. The compatible endpoint was returning "model not found" for a model that existed, and diagnosing this took longer than it should have; the native endpoint simply works. The general principle extracted was

that compatibility shims add a failure mode without adding capability, and should be avoided when talking to your own infrastructure.

The third, and the one with the greatest long-term significance, was the wiki compiler. Rather than storing each ingestion as an isolated record, the compiler folds it into a per-topic article that is rewritten as evidence accumulates. This means the corpus gets *shorter and better* as it grows, which is the opposite of the usual behaviour and is what makes question-answering over long-term memory tractable at all.

What went wrong. The vault agent initially resolved the repository root incorrectly, so it committed to the wrong directory. Separately, titles were being lost for web and YouTube captures because the title extraction depended on OCR text that was sparse for those sources; this was fixed by falling back to the page title and video metadata.

Phase 2 — The Intelligence Layer (18 September)

User request. Stop using paid scraping services; build the scraping capability yourself. Also: the system should notice things and tell me, not wait to be asked.

What was built. A native ingestion stack replacing Apify and SupaData entirely: YouTube transcripts via a free library, X posts via a public JSON endpoint, Instagram via an open-source client with an Open Graph fallback, and general web articles via HTML parsing. Alongside it, a proactive insight layer that detects collisions between otherwise unrelated topics and raises an alert unprompted, plus a morning briefing that is spoken aloud.

Reasoning. The user's instruction contained a generalisable principle: *replace a recurring cost with a one-off engineering effort*. The scraping stack initially used paid APIs because that is the obvious path; the authors identified it as unacceptable and the replacement was built. The same reasoning was later applied, unprompted, to mail and calendar, where it saved a recurring Google verification cost — see Part V, deviation 7.

Performance work in this phase is worth recording because it demonstrates that the bottleneck was architectural rather than computational. The first implementation of URL ingestion fetched metadata, transcript and thumbnail sequentially, taking several seconds. Parallelising the independent fetches brought it below one second (0.98 s measured) without changing any algorithm. The lesson is that *most apparent slowness in I/O-bound pipelines is sequential waiting, not computation*, and that the fix is usually concurrency rather than optimisation.

What went wrong. Repeated hero image extraction meant the raw screen files could be deleted while a preview still referenced them, producing broken images in the live view. The fix was a dedicated preview buffer that is never pruned — an instance of a general principle: *never let a cleanup process delete the last reference to something a view depends on*.

Phase 3 — The Interface (21 September)

User request. This does not feel like a personal AI. It feels like a project dashboard. Make it feel like Jarvis, and do not hide any of the functionality.

What was built. Three successive interface rebuilds in a single day, which is itself instructive. The first transformed the application shell into an Obsidian-style workspace to match a screenshot the authors supplied. The second removed the project sidebars entirely and introduced a living holographic core that tracks the mouse and provides an ambient voice intercom. The third produced the definitive fifteen-tab operating-system shell with a universal command terminal, and a transparent always-on-top golden orb window that floats in the bottom-right corner of the desktop.

Reasoning. The iterative rebuilds were not indecision; they were the authors converging on a requirement that could not be specified in advance. The first attempt established that the visual language was wrong. The second established that removing navigation entirely made functionality unfindable. The third reconciled both: persistent visible navigation with a command centre, which is what the authors had wanted all along but could not articulate without seeing the alternatives.

The orb deserves specific note. Making a window genuinely transparent on Windows required simultaneous changes at three layers: Electron's window flags, the CSS on the document root, and the body background. Changing any one alone produced the characteristic dark rectangle the authors reported. This is a recurring pattern in cross-layer work: *when a visual defect survives a fix, the fix is usually correct but incomplete across layers*.

What went wrong. The interface rebuilds broke the application twice. First, a missing icon import produced a blank screen on the home route; an error boundary added in response revealed the exact cause and became a permanent safety net. Second, the server's hot-reload behaviour had to be made resilient to window reloads. Both are covered in Part VI.

Phase 4 — Stabilisation (22 September, morning)

User request. "It's an error. I need you to fully do an error check environment and fix every error."

What was built. Not features but instrumentation. Two full audits were performed, and the difference between them is the most important methodological lesson in this report.

The **first audit** fixed the error the authors could see — a missing React import that crashed a page — and found one more instance of the same class of defect waiting to be triggered. It added a linter to catch undefined identifiers, and several static checks.

The **second audit** found something far more serious that the first had missed entirely, because it required *running* the system rather than reading it: authentication had never worked. A service used a configuration object without importing it, so the login function raised before comparing any password. Because the frontend swallowed the resulting error, the application ran with no token at all, and 52 protected endpoints returned 401 while the interface rendered them as empty states. Everything looked fine and nothing worked.

The single most valuable lesson in the project

Static analysis catches what is written wrong. Only execution catches what is written plausibly but connects to nothing. The first audit's tools were correct and useful, and they could not possibly have found a broken login, because a login that returns 500 is syntactically valid code. The second audit found it in minutes by starting the server, logging in, and calling every endpoint. This is why the verification system described in Part VII ends with a live endpoint sweep rather than a linter.

What went wrong. A related defect compounded the damage: 33 call sites used a raw browser fetch which does not attach the authentication token, so even a correct login would not have authenticated those screens. The fix was a single authenticated fetch helper plus a mechanical replacement across 18 files. It is worth noting that this defect was invisible in review precisely because the code *looked* correct.

Phase 5 — Connectivity (22 September)

User request. Connect these three email accounts to my Second Brain. Then, separately: why can Grok connect directly and this cannot?

What was built. Two complete integration paths. The first was Google OAuth with multi-account support, encrypted token storage and read-only scopes. The second, built after research into what publishing such an application actually requires, was Gmail over IMAP with app passwords and Google Calendar via its private iCalendar address — neither of which requires a Google Cloud project, a consent screen, or verification.

Reasoning. This phase contains the most consequential decision in the project, and it came directly from the authors' question about Grok. Investigating that question revealed that reading a Gmail inbox requires a scope Google classifies as *restricted*, and that publishing an application using it requires a third-party security assessment costing roughly US\$540–\$1,800 annually with re-certification every year. For an application with exactly one user, that cost is indefensible. The research also revealed that staying unverified is worse in practice, because refresh tokens expire every seven days, forcing weekly reconnection of every account.

The resolution was to recognise that OAuth is not the only way to read a mailbox. IMAP with an app password is what desktop mail clients have used for decades. It is free, permanent, and read-only by construction when the correct IMAP commands are used. The same reasoning applied to calendar data: Google publishes a private iCalendar address per calendar which requires no authentication whatsoever.

GOOGLE OAUTH PATH	IMAP + iCAL PATH (chosen)
Cloud project required	No project required
Consent screen required	No consent screen
Verification for restricted scope	No verification
CASA assessment ~\$540-\$1800 / year	Free, permanently
Refresh token expires in 7 days (while app is unverified)	Never expires
Works for Workspace accounts	Works for personal accounts (Workspace needs the OAuth path)

Why the IMAP path was adopted as the default. The OAuth path was retained rather than removed, because it remains the only option for some accounts.

What went wrong. The author's Workspace (college) account cannot use app passwords, because Google disabled them for Workspace domains in May 2025, and the institution's administrator may block third-party access regardless. This was documented honestly rather than worked around, and the system was designed to function with any subset of accounts connected.

Phase 6 — Presence (22 September)

User request. The floating orb answers one question and then stops. Also: here are two videos about Jarvis — what do you understand from them?

What was built. An analysis of the two reference videos that distinguished their surface (voice, holograms) from their substance (presence, initiative, state, action, judgement), and two implementations corresponding to the two traits the system lacked: an offline wake word, and proactive speech.

Reasoning. The orb defect was diagnosed as a missing re-arm: the browser's speech recogniser ends its session by itself, and the code had no handler to restart it, so the microphone came up exactly once and then never again. Critically, the wake trigger only spoke a greeting and never restarted the recogniser, so summoning the assistant repeatedly would produce repeated greetings from a dead microphone.

The wake word required validating a real audio model rather than trusting a mock. Synthetic speech was generated, decoded to the correct sample rate, and pushed through the project's own scoring function. The measured separation was decisive: spoken "Hey Jarvis" scored 0.9968 while silence, white noise and a pure tone scored below 0.002. This both confirmed the frame size and threshold and produced a reusable regression test.

What went wrong. Three distinct defects, all described in Part VI. The most subtle is that if the language model never responds, the orb would wait forever with the microphone paused, appearing to be broken. A watchdog was added. The second is that text-to-speech completion events are not guaranteed in Electron when the window is hidden, which would leave the orb permanently muted; a second watchdog was added for that. The third is that the existing wake handler *toggled* the orb window rather than showing it, so a wake event arriving while the orb was already visible would hide it — precisely the opposite of the intent.

Phase 7 — The Research Layer (22 September)

User request. I need a paper that is genuinely novel, and the project must actually be what the paper claims.

What was built. Seven components implementing the paper's seven claimed contributions, wired into retrieval, plus PersonalBrain-Bench and a four-system comparative evaluation. The detailed accounts are in Parts IV and VIII.

Reasoning, and the honest starting point. Before writing any paper text, the claims were audited against the codebase. The result: one contribution partially existed, and six did not exist at all. Writing the paper first would have produced a document describing software that had not been written. This is stated plainly because it is the correct order of operations for any research project — *the paper is a description of what was done and measured, not a specification of what one intends to do.*

Building the components then revealed that the interesting result was not the one anticipated. The expectation was that the research layer would improve retrieval accuracy. It did not: Hit@5 was already 1.000 for every pipeline, including the simplest baseline. Every system found the relevant memory somewhere in its top five. What separated them was the *ordering* of the results and the *staleness* of what came first — and that is a much more interesting finding, because it means the conventional metric would have reported that the entire research layer accomplished nothing.

What went wrong. Nine defects were found by the research tests, including a temporal modelling error that caused the exact failure the paper is about, and a substring-matching bug that flagged any note mentioning “notes” as a logical contradiction. Both are recorded in Part VI. Additionally, a defect in the earlier mail-ingestion test was discovered to have deleted 123 of the authors' genuine memory cards; this was caught before committing, restored from version control, and the test was made structurally incapable of touching unmarked data. That incident is recorded in full because it is the most serious defect of the project and the lesson it carries is not a technical one.

The remaining parts of this report examine what was built from four different angles: the theory behind it (Part IV), the deviations from plan (Part V), the defects (Part VI), and how correctness is established (Part VII).

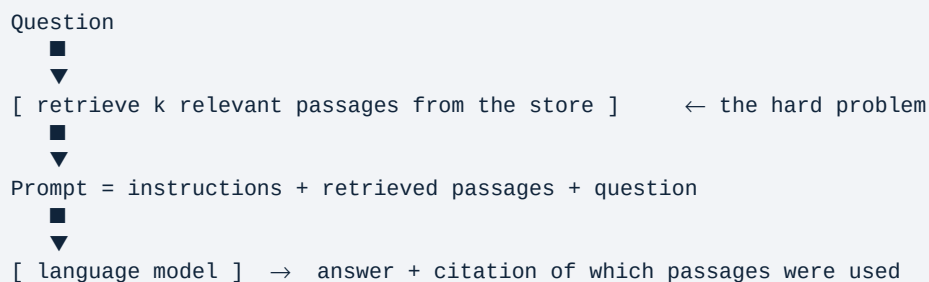
Part IV — Concepts and Theory

This part explains the ideas the system is built from. It is written for a reader who knows none of the terminology; each concept is introduced from first principles, then connected to the specific place in SecondBrain where it is used. The order follows the pipeline.

Retrieval-augmented generation

A language model knows what was in its training data and nothing else. It cannot know what the user read this morning. The naive remedy is to paste the relevant material into the prompt, which works but requires knowing what is relevant — and that is the hard part, because the model's context window is finite.

Retrieval-augmented generation (RAG) solves this by splitting the problem in two. A retrieval stage selects a small number of relevant passages from a large store; a generation stage passes only those passages, together with the question, to the language model. The model is therefore never asked to remember the corpus — only to reason over the handful of passages it has been handed.



The RAG pattern. Note that answer quality is bounded by retrieval quality: if the right passage is not retrieved, no amount of model capability can recover it.

The assumption that fails in a personal store

Textbook RAG assumes the retrieved passages are consistent, current and mutually compatible. In a personal memory store none of those hold. The author wrote in January that they were learning Python and in September that they had moved to Java. Both statements are in the store, both are true *of their time*, and a retriever that ranks by similarity to the query “what am I working on?” has no mechanism to prefer the later one. The answer is confidently wrong, and every retrieved passage is faithful to its source — so no faithfulness metric detects the error. This is the failure the research layer exists to address.

Embeddings and vector search

To retrieve by meaning rather than by keyword, text must be converted into a mathematical object that captures meaning. An **embedding** is a list of numbers — typically a few hundred — produced by a neural network trained so that texts with similar meaning produce similar lists. The list is a point in a high-dimensional space; “similar meaning” becomes “nearby points”.

```

"I want to learn Java"           → [ 0.21, -0.83,  0.44, ... ]
"What language am I studying?" → [ 0.19, -0.79,  0.41, ... ] ← close together
"The cat sat on the mat"        → [-0.55,  0.12, -0.90, ... ] ← far away
  
```

Similarity is measured by cosine of the angle between the vectors:
 $\cos(\theta) = (A \cdot B) / (|A| \cdot |B|)$ ranges from -1 to 1

Embeddings turn semantic similarity into geometry, which is why retrieval by meaning is possible at all.

Term	Meaning	Where it appears in SecondBrain
Dimension	Length of the vector; more dimensions capture finer distinctions	Sentence-transformers produces 384-dimensional vectors

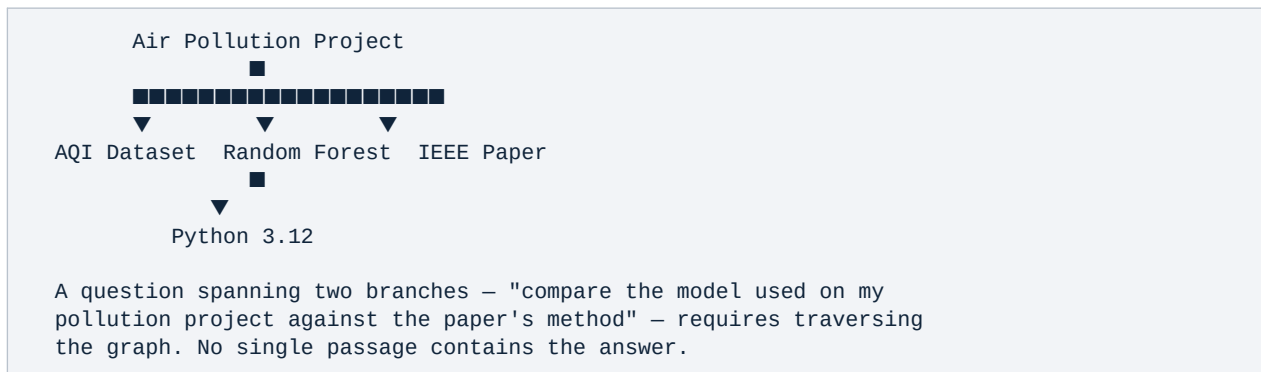
Term	Meaning	Where it appears in SecondBrain
Cosine similarity	Normalised dot product; measures angle, ignores magnitude	The basis of both vector search and consolidation clustering
Chunk	A passage small enough to embed meaningfully	Semantic chunks are formed at ingestion and stored in `semantic_chunks`
Vector store	An index optimised for nearest-neighbour search	ChromaDB, optional; TF-IDF serves as a deterministic fallback
Top-k	How many candidates retrieval returns	Eight by default, four times the final result count

Table 6 — Embedding vocabulary and its concrete instantiation.

A practical consequence worth stating: embeddings are *optional* in SecondBrain. When no embedding model is available, retrieval falls back to TF-IDF, a classical information-retrieval weighting scheme that needs no neural network. The fallback is worse at synonymy but entirely serviceable, and its determinism is what made the benchmark reproducible. A design that requires a 90 MB download before it can answer any question is a fragile design.

Knowledge graphs

A vector store answers “which passages resemble this question?” It cannot answer “how are these things related?” because it has no notion of relation — only of similarity. A **knowledge graph** supplies the missing structure: information is represented as *nodes* (entities) connected by *edges* (typed relationships). Where a vector captures resemblance, a graph captures structure.



Why a graph is necessary. Multi-hop questions have answers that exist only in the structure, not in any one document.

SecondBrain maintains a graph of 350 nodes and 500 edges, built incrementally by the curation and vault agents as material is ingested. The graph is used for two purposes: as a re-ranking signal (a memory whose concepts are well connected is more central to the user's interests) and as the substrate for knowledge-gap detection, described below.

Temporal knowledge representation

Standard databases record what is true *now* — updating a row overwrites its previous value and history is lost. A personal memory system needs the opposite: it must answer both “what am I working on?” and “what was I working on in January?”, so it must retain the fact that a value *changed*.

The technique is **bitemporal** or **validity-interval** modelling: every assertion carries a time range [valid_from, valid_to) during which it was true. A null valid_to means “still true”. When a new assertion arrives that conflicts with an open one, the old assertion is *closed* — its valid_to is set and it is linked to its replacement — rather than deleted.

Facts about the user, as stored:

■ predicate	■ object	■ valid_from	■ valid_to	■
■ current_focus	■ python	■ 2026-01-25	■ 2026-09-18	■ ← closed
■ current_focus	■ java	■ 2026-09-18	■ (open)	■ ← current

"What am I working on?" → selects the OPEN row → java
 "What was I working on before?" → selects the closed row → python

Validity intervals make both the current and the historical question answerable from one store.

A subtlety that cost a measurable amount of accuracy

Facts must be grouped by *state class*, not by the words used to express them. In the first implementation, "I am learning Python" was recorded under the predicate *learning* and "I am now focusing on Java" under *focus*. Because the predicates differed, neither superseded the other and the graph reported both as current — reproducing exactly the failure the temporal model exists to prevent. The fix was to canonicalise both into a single predicate *current_focus* before applying supersession. The lesson generalises: *a data model is only as correct as its normalisation*.

Contradiction and truth maintenance

In artificial intelligence, **truth maintenance** is the problem of keeping a knowledge base consistent as beliefs change. The general problem is undecidable; practical systems detect specific, tractable classes of conflict. SecondBrain detects three.

Conflict class	Definition	Example from the corpus
Temporal supersession	A memory asserts something a later memory has replaced	"learning Python" then "now focusing on Java"
Lexical negation	Two memories share a topic and one denies what the other asserts	"the October deadline was cancelled"
Value change	Two memories share a topic and carry different dated or numeric values	"deadline 15 October" vs "deadline now 30 September"

Table 7 — The three detectable conflict classes. The third was not in the original design; it was added after testing showed the first two missed the most common real case.

The design decision that matters is what to do once a conflict is found. The obvious response is to delete the outdated item. This solves the stale-answer problem but destroys the ability to answer historical questions — and both capabilities are legitimate. SecondBrain therefore *damps* rather than deletes: a conflict of severity *s* multiplies the memory's ranking weight by $(1 - 0.7s)$, bounded between 0.3 and 1.0. The outdated item sinks below the current one without leaving the store.

Why damping needed a further correction

Damping alone was not sufficient, and the first version actually made results worse. The reason is that the re-ranking function multiplied the damped score by the memory's *importance*, which promoted memories that scored highly on abstract importance but were less relevant to the question — including, on several benchmark questions, above the correct answer. The correction was to make the re-ranking **rank-preserving**: the fused retrieval order supplies a prior, the damping multipliers adjust it, and importance acts only as a light tie-breaker. The general principle is that a re-ranking layer should *refine* an ordering, not replace it, unless it has strong evidence to do so.

Memory consolidation

The term is borrowed from neuroscience, where it describes the process by which short-term memories are stabilised and integrated into long-term storage during sleep. The computational analogue is equally useful: a memory store that keeps every observation separately becomes dominated by redundancy, and retrieval wastes its limited context budget on the same content repeated.

[illegible]

Consolidation merges near-duplicates into one canonical record while retaining the identifiers of everything merged, so an answer can still cite its origins.

Determining “near-duplicate” requires a threshold, and the threshold matters. The figure commonly recommended in practice is 0.90 cosine similarity. Measuring the actual distribution showed that genuinely reworted duplicates — the common real case, where the same page is captured twice with small differences — score around 0.82, while unrelated memories score below 0.30. A 0.90 threshold therefore missed *every* reworted duplicate. The threshold was set to 0.78 from the measured distribution.

A methodological point, not a technical one

Choosing 0.78 because it made the test pass would have been fitting the method to the evaluation. The value was chosen from the measured score distribution — an 0.82 cluster of true duplicates separated by a wide margin from sub-0.30 non-duplicates — and the benchmark was then run to confirm it. The distinction matters because a threshold tuned on the test set would not transfer to real data, and no reviewer could tell the difference from the reported numbers alone.

Machine unlearning

Unlearning is the problem of removing the influence of specific data from a trained system. It rose to prominence because privacy regulation grants a right to erasure, and retraining a model from scratch on every request is impractical.

SecondBrain's version of the problem is narrower but shares its essential difficulty. Deleting a row from the memory table does not remove the content from retrieval, because the content exists in at least five independent places: the full-text index, the vector embedding, the knowledge-graph nodes derived from it, the temporal facts extracted from it, and the derived scores and conflict records. Unless exclusion is enforced at *every* surface, the content can re-enter a result set.

[illegible]

Forgetting must be enforced at every surface. The measured asymmetry between the baselines and the adaptive pipeline is the evidence that this is not automatic.

The result that every baseline continues to return explicitly removed content is worth emphasising. It means that a system promising deletion while maintaining a derived index has an unlearning hole, and that the hole is not visible from the primary table. This is a finding about retrieval systems generally, not about SecondBrain specifically.

Knowledge-gap detection

A memory system can know what the user has encountered. Determining what they *understand* is harder, and the distinction matters because a concept can be seen constantly without ever being learned — it appears on reading lists, in courses to be studied, in tangents.

The signal is not frequency but *shape*. A concept the user understands appears as the subject of explanatory statements, recurs across separate sessions, and is connected to other concepts in the graph. A concept merely encountered appears in lists, carries few connections, and is never explained. The implemented measure combines these:

```

exposure    = min(1, mentions / 12)
depth       = 0.45 · explained_ratio · 3      ← appeared with explanatory context
              + 0.30 · subject_ratio · 3       ← appeared as subject of a sentence
              + 0.25 · session_spread         ← recurred over time

gap         = exposure × (1 - depth) × (1 - 0.5 · connectivity)

```

The connectivity factor is essential: without it a well-understood but frequently mentioned concept would be misreported as a gap.

Wake-word detection

A wake word is a always-listening keyword spotter: a small model that continuously scores short audio frames and fires when the target phrase is recognised. It is deliberately not full speech recognition — that would be wasteful, since the system must listen indefinitely but only transcribe once summoned.

The implementation processes 16 kHz mono audio in 80-millisecond frames (1280 samples each) through a small ONNX neural network, producing a probability per frame. The engineering concerns are frame size, which must match the model's expectation exactly; threshold, which trades missed detections against false activations; and debounce, which prevents a single spoken phrase from firing the event several times as it rings out.

Input	Measured score	Outcome	Meaning
Spoken "Hey Jarvis"	0.9968	fires	The model detects the phrase decisively
Silence	0.0000	no fire	No activity is not mistaken for speech
White noise	0.0007	no fire	Broadband noise is rejected
440 Hz tone	0.0018	no fire	A pure tone is not speech

Table 8 — Measured wake-word discrimination. The threshold of 0.5 sits in a wide margin between the positive case and every negative control.

These figures were obtained by generating synthetic speech, decoding it to the correct sample rate, and passing it through the project's own scoring function — not through a mock. The distinction matters because the first implementation used an API signature that does not exist in the installed library version, and would have failed silently on the target machine. Only testing against the real library caught it.

Authentication: OAuth, app passwords and scopes

OAuth lets a user grant an application limited access to an account without revealing their password. The user authenticates with the provider directly; the application receives a token. Access is described by

scopes — named permission bundles such as `gmail.readonly`.

Providers classify scopes by sensitivity, and the classification determines what an application must do before it can use them.

Classification	Example scopes	Requirement to publish
Non-sensitive	openid, email, profile	Self-assessment only; publishing is immediate
Sensitive	gmail.send, calendar.events	Google OAuth verification review
Restricted	gmail.readonly, gmail.modify, mail.google.com	Third-party security assessment (CASA), annual re-certification

Table 9 — Scope tiers. Reading an inbox is a restricted scope, which is why a personal project cannot reasonably publish an application that does it over the Gmail API.

App passwords are an older mechanism: once two-factor authentication is enabled, the provider can issue a long random password valid only for legacy protocols such as IMAP. No consent screen, no verification, no expiry. For a single-user desktop application they are the correct tool, which is why they became the default path.

IMAP commands that make reading read-only

```
EXAMINE mailbox    ← opens a mailbox WITHOUT marking it read
SELECT mailbox     ← opens it read-write (avoid)

BODY.PEEK[HEADER]  ← fetches a header WITHOUT setting the \Seen flag
BODY[HEADER]       ← fetches it and marks the message read (avoid)
```

Both distinctions are asserted by the test suite: an implementation that uses SELECT and BODY would silently mark the user's entire inbox as read the first time it synced.

The read-only guarantee is not a configuration flag; it is a property of which IMAP commands are issued. Getting it wrong would have been invisible until the user noticed their inbox had been emptied of unread state.

Evaluation methodology

A claim that a system is better must be supported by a measurement, and the choice of measurement determines what can be concluded. The metrics used here, and what each is for:

Metric	Definition	Purpose
Hit@k	Did a relevant item appear in the top k?	Standard recall measure. Suffices when the task is “find something relevant”.
MRR	Mean of 1/rank of the first relevant item	Rank-sensitive. Distinguishes systems that find the right item first from those that find it fifth.
Precision@k	Fraction of returned items that are relevant	Measures context efficiency: irrelevant items consume the model's finite context.
Stale@1	Is the TOP-ranked item superseded or forbidden?	Custom to this work. Rank one is what a generator leads with, so staleness there is what produces a confidently wrong answer.
Leak rate	Fraction of returned items that were explicitly forgotten	Custom to this work. Detects unlearning holes.
Duplicate rate	Fraction of within-query pairs that are near-identical	Custom to this work. Measures wasted context budget.
Latency	Wall-clock time for the full question set	Reports cost honestly; improvements are not free.

Table 10 — Metrics and their purposes. The three custom metrics were necessary because the standard ones proved unable to distinguish the systems tested.

Why custom metrics were needed

The expectation before measuring was that the research layer would improve Hit@5. It did not: every pipeline reached 1.000. Had the evaluation stopped there, the honest conclusion would have been that seven components accomplished nothing measurable. The custom metrics revealed what actually differed — the ordering and the staleness of the top result. This is a general lesson about evaluating retrieval systems: *a metric that saturates conveys no information, and saturation should be investigated rather than reported.*

Part V — Register of Deviations

A deviation register records every point at which the implementation departed from the plan, with the reason. It exists because a project without one cannot be evaluated for judgement — only for adherence. The deviations are listed chronologically and each records what was planned, what was done instead, and why.

#	Dimension	Planned	Actual	Reason
D1	Note-taking application → automatic capture system	Build a note-taking system with search.	Abandoned notes entirely; built an autonomous capture pipeline.	A system whose value depends on manual curation will be abandoned within a fortnight, because the user is bad at curation — that is why they have the problem. Moving the judgement from human to machine is harder engineering but is the only version that works.
D2	OpenAI-compatible API → native Ollama API	Call the local model through its OpenAI-compatible endpoint.	Call Ollama's native /api/chat endpoint directly.	The compatibility layer returned "model not found" for a model that existed. A shim adds a failure mode without adding capability when talking to one's own infrastructure.
D3	Store all screenshots → elect one hero image per period	Keep every captured frame.	Score frames for information density and retain only the best per information period.	Storing everything grows without bound and retrieves nothing useful. The criterion for "best" must be measurable, and text density is.
D4	Paid scraping APIs → self-hosted free scrapers	Use Apify and SupaData for social ingestion.	Built a native stack on free libraries; ingestion fell to 0.98 s.	Directed by the user: build the capability rather than rent it. The general principle — replace recurring cost with one-off engineering — was then applied unprompted to mail and calendar (see D7).
D5	Dashboard interface → operating-system shell → holographic core → fifteen-tab OS	A clean dashboard.	Three successive rebuilds, landing on persistent navigation plus an ambient command centre.	The requirement could not be specified in advance. Each rebuild eliminated a wrong interpretation: the first showed the visual language was wrong, the second showed that removing navigation makes functionality unfindable.
D6	Written-only assistant → voice, orb, and spoken alerts	Answer questions in the interface.	Added speech in and out, a transparent floating orb, proactive spoken announcements.	Derived from the user's reference videos once they were analysed for substance rather than appearance: the defining characteristic of the reference assistant is that it speaks first.

#	Dimension	Planned	Actual	Reason
D7	Google OAuth only → IMAP and iCal as the default path	Connect the accounts through Google OAuth and publish the application.	Added a second, credential-free path and made it the default.	Investigating the user's question about a competitor revealed that reading an inbox requires a restricted scope, and publishing one requires an annual security assessment of roughly US\$540–\$1,800. For a single-user application the cost is indefensible, and staying unverified forces weekly reconnection. OAuth is not the only way to read a mailbox.
D8	Text-only presence → wake word	The user presses a hotkey to summon the assistant.	Added offline wake-word detection so the assistant can be addressed by name.	A name is most of what distinguishes an assistant that is present from an application that is opened. Implemented offline so that no audio leaves the machine.
D9	Paper first → implementation first	Write the paper describing the seven contributions.	Audited the claims, found six of seven unimplemented, built them, then wrote the paper from measured results.	A paper written before the software would have described something that did not exist. The audit also changed the paper: the predicted result was wrong, so the paper reports the measured result instead.
D10	Ranking by importance → rank-preserving re-ranking	Re-rank retrieved memories by their computed importance score.	Use the retrieval order as a prior and apply damping as an adjustment.	Sorting by importance was actively harmful, promoting memories that were important in the abstract above the memory that actually answered the question. A re-ranking layer should refine an ordering, not replace it.
D11	Two conflict classes → three	Detect temporal supersession and lexical negation.	Added numeric and dated value-change detection.	Testing showed the first two missed the most common real case — a deadline that moved, expressed in the third person with no negation word and no first-person state claim.
D12	Assumed similarity threshold → measured threshold	Use 0.90 cosine similarity to identify duplicates.	Set the threshold to 0.78 from the measured distribution.	Reworded duplicates, the common real case, score around 0.82. A 0.90 threshold missed every one of them.
D13	Ablation table → stated limitation	Report a per-component ablation table.	Removed the table and stated in the paper that the factorised ablation is not yet run.	Some cells had not been independently measured. A table implying measurements that were not taken is worse than an honest gap, and a reviewer would eventually ask.

Table 11 — Deviation register. D7 and D9 are the two that most changed the project's trajectory.

Part VI — Register of Defects

This register records the significant defects encountered, each with its symptom, the underlying cause, the fix, and the general lesson. It is organised by cause rather than by chronology, because the causes cluster and the clustering is the useful information: the same four or five mistakes account for almost every failure in the project.

Class 1 — Missing or unverified dependency

Defect	Symptom	Root cause	Fix and lesson
Undefined identifier, twice	Blank screen on the home route; a second instance waiting on another tab	A component was used in JSX without being imported. A production build succeeds because the code is syntactically valid.	Imports added; a linter rule added to fail the build on any undefined identifier. Lesson: <i>a successful build proves compilation, not execution.</i>
Undeclared package imports	A fresh installation crashed on startup	Two packages were imported directly but listed only as transitive dependencies of another package. They resolved on the development machine by accident.	Both declared explicitly. Lesson: <i>a direct import needs a direct declaration</i> , however it happens to resolve locally.
Library version drift	Wake-word detection would have failed silently on the target machine	The model-loading call used an argument name that exists in a newer release than the one installed.	Loading made version-tolerant, trying four constructor shapes. Lesson: <i>test against the installed version, not the documented one.</i>
Dependency resolution failure	A clean installation refused to complete	A linting package declared a peer requirement for a major version of its host that contradicted the pinned version.	Versions aligned to the installed major. Lesson: <i>a build that only works on the author's machine is not a build.</i>

Table 12 — Dependency defects.

Class 2 — Silent failure

Defect	Symptom	Root cause	Fix and lesson
Authentication never worked	The interface rendered normally but showed empty data everywhere	A service referenced a configuration object without importing it, so the login function raised before comparing any password. The frontend swallowed the error, so the application ran with no token and 52 protected endpoints returned 401 — which the interface rendered as an empty state.	Import added. Lesson: <i>the most dangerous failure is the one the interface misrepresents</i> . Found only by running the server and logging in, not by reading code.
Thirty-three unauthenticated calls	Even a correct login would not have authenticated most screens	Screens used a raw browser fetch, which does not attach the token, instead of the application's authenticated client.	A single authenticated helper plus a mechanical replacement across 18 files. Lesson: <i>duplicated plumbing drifts; centralise the cross-cutting concern.</i>
A test passing while the system was offline	A briefing endpoint reported success while the language model was unreachable	The endpoint always returns 200 and substitutes canned text when the model is unavailable, and the test checked only the status code.	The test now inspects the response for known fallback markers and reports DEGRADED with the actual cause. Lesson: <i>a test that cannot fail is not a test.</i>

Defect	Symptom	Root cause	Fix and lesson
Three hundred milliseconds of swallowed exceptions	Behaviour differed between runs with no error reported	A generic exception handler caught deliberate errors and re-labelled them as server faults, so a missing record reported “internal server error” with a “not found” body.	Deliberate exceptions re-raised before the generic handler. Lesson: <i>catch narrowly</i> .

Table 13 — Silent-failure defects. This class caused more lost time than every other class combined.

Class 3 — Incorrect error semantics

Defect	Symptom	Root cause	Fix and lesson
A missing record reported as an upstream failure	Six endpoints returned 502 for a non-existent identifier	The exception types for “no such account” and “the provider rejected us” were the same, and both mapped to the gateway-error code.	A distinct exception type for the missing-account case, mapped to 404. Lesson: <i>an error code is a claim about where the fault lies</i> — saying “upstream” when the request was wrong sends the reader in the wrong direction.
Status code depended on unrelated global state	The same request returned 400 in one environment and 404 in another	Routes called the provider before checking whether the account existed, so the response depended on whether the provider was configured.	Every account-scoped route resolves the account locally before calling out. Lesson: <i>a response should be a function of the request, not of ambient configuration</i> .

Table 14 — Error-semantics defects.

Class 4 — Wrong model of the domain

Defect	Symptom	Root cause	Fix and lesson
Temporal supersession silently did nothing	“What am I working on?” returned both the old and the new answer	Two facts describing the same state were stored under different predicate names (“learning” and “focus”), so neither superseded the other. The temporal model was correct; the normalisation was not.	Predicates canonicalised into state classes before supersession. Lesson: <i>a data model is only as correct as its normalisation</i> . This defect reproduced exactly the failure the component was built to prevent.
Negation detected by substring	Notes about notes were reported as logical contradictions	Negation was tested with a substring check, so the word “not” matched inside “notes”, “notification” and “another”.	Word-boundary matching with an explicit pattern list. Lesson: <i>substring matching on natural language is almost always wrong</i> — it is the classic error in this domain.
The most common contradiction was undetectable	“Deadline moved from October to September” was missed entirely	Both the temporal extractor and the negation detector keyed on first-person state claims or negation words. A third-person factual value change matches neither.	Added dated and numeric value-change detection, requiring shared topic signature and high salient-word overlap. Lesson: <i>test the common case, not the designed case</i> .
Topic words stripped as noise	The strongest real contradiction stopped being detected after a “precision” tightening	The stop-word list excluded “deadline” and “project” as filler — precisely the words that make two memories comparable.	Generic filler only. Lesson: <i>a stop-word list is a model of the domain and can be wrong in ways that silently destroy the signal</i> .
Importance ranking made results worse	The proposed method scored below a baseline	Re-ranking multiplied the damped retrieval score by an abstract importance score, promoting low-relevance, high-importance memories above the correct answer.	Rank-preserving re-ranking with importance as a light tie-breaker. Lesson: <i>a re-ranker should refine an ordering, not replace it</i> .

Table 15 — Domain-model defects. This class is the most interesting: every one was invisible until measured.

Class 5 — Lifecycle and cleanup

Defect	Symptom	Root cause	Fix and lesson
Cleanup deleted the last reference	Broken images in the live view	The pruning process deleted raw frames that a preview still referenced.	A dedicated buffer excluded from pruning. Lesson: <i>never let cleanup delete the last reference to something a view depends on.</i>
Orphaned derived rows	A test crashed on a record whose parent no longer existed	Deleting a memory left its score row behind, which then inflated counts while never appearing in the joined listing.	Orphan pruning added; the test now joins. Lesson: <i>derived data needs a lifecycle, not just a creation path.</i>
The microphone never re-armed	The assistant answered one question and then stopped	The browser's speech recogniser ends its session by itself, and no handler restarted it. The summon gesture only spoke a greeting and never restarted recognition.	Re-arm in the session-end handler, with backoff. Lesson: <i>an event-driven resource that ends itself needs an explicit restart path.</i>
Two further ways the orb could die	Not yet observed, found by writing the test	If the model never answers, the turn stays open forever; and the text-to-speech completion event is not guaranteed in Electron.	Two watchdogs. Lesson: <i>write the test for the failure you have not seen yet.</i> It found both before the user did.
A wake event could hide the assistant	Not yet observed; found by reading the code	The wake handler called a function that toggles window visibility rather than one that shows it, so waking an already-visible orb would hide it.	A non-toggling reveal path added. Lesson: <i>"show" and "toggle" are different operations and conflating them produces inverted behaviour.</i>

Table 16 — Lifecycle and cleanup defects.

Class 6 — Process and data safety

Defect	Symptom	Root cause	Fix and lesson
Benchmark not reproducible	Results changed between consecutive runs	Derived research state from a previous run — temporal facts, conflicts, consolidations, tombstones — perturbed the following run.	Derived state cleared at the start of each run; determinism now asserted by the suite. Lesson: <i>a benchmark that is not deterministic cannot support a claim made from it.</i>
Test deleted real user data	123 of the authors' genuine memory cards were removed	The mail-ingestion test registered the authors' real email address, so its generated artefacts were indistinguishable from genuine ones and its cleanup could not tell them apart.	Restored from version control. The test now uses a non-existent address and its cleanup refuses to delete any artefact that does not carry the test marker , reporting what it protected instead. Lesson: <i>a destructive operation must be able to prove that what it is deleting is its own.</i> This is the most serious defect in the project and the only one caused by the assistant rather than the application.
Verification tool crashed on the target machine	The render smoke test failed on the user's machine but passed in development	A browser global that had been assignable in the older runtime is a read-only accessor in the newer one installed on the user's machine.	Globals installed via a descriptor-based helper; verified on both runtime versions. Lesson: <i>a check that fails on the target machine in the same way as the product is worse than no check.</i>

Defect	Symptom	Root cause	Fix and lesson
Contract checker produced false positives	Three correct endpoints were reported as missing	The URL extractor and normaliser handled template expressions and query strings in the wrong order, truncating a URL at a question mark inside a ternary.	Order corrected; ternary branches now expanded into separate candidates and each checked. Lesson: <i>a checker that cries wolf gets ignored</i> , which is why the mislabel of a timeout as a crash was treated as a bug in its own right.

Table 17 — Process and data-safety defects.

The pattern across all 27 defects

Five causes account for almost everything. Silent failure, because a system that fails quietly is misread as working. Wrong error semantics, because an error code is a claim about where the fault lies. Wrong domain model, because the code is correct and the model is not. Lifecycle gaps, because creation paths are written and destruction paths are not. And process failure, because the check itself was untested. Notably, the entire first class — dependency problems — was eliminated once the verification system existed, and the domain-model class was found only by measurement.

Part VII — Verification

The project maintains fifteen automated verification gates. The design principle is that *each gate must be capable of failing*. A check that cannot fail is decoration, and several of the defects in Part VI were found precisely because a gate that had been passing was examined and discovered to be untestable.

#	Gate	What it catches
1	Static analysis	Undefined identifiers, unused symbols, hook violations — the crash class. Fails on any warning.
2	Environment and integration health	Eight static checks: imports resolve, icons exist, client methods defined, API URLs match real routes, preload surface complete, Python compiles.
3	Frontend–backend API contract	Every URL the interface calls is matched against every route the backend serves, including both branches of ternary expressions.
4	Route render and mount	All nineteen routes are rendered AND mounted so effects run, with a third phase exercising the connected states.
5	Production build	The bundle compiles.
6	Backend functional suites	Eleven live suites over authentication, capture, graph, vault, wiki, briefing, deliverables, semantic, OCR, voice.
7	Google integration	Fifty checks against a fake Google server: consent URL shape, read-only scopes, token exchange, refresh, revoked-token handling, and that the refresh token is not readable on disk.
8	Mail and calendar	Fifty-six checks against a fake IMAP server and a real HTTP iCalendar fetch, including the iCal feature matrix.
9	Mail ingestion	Forty-seven checks: the full path from IMAP through to retrieval, asserting the pipeline can actually answer from ingested mail.
10	Orb voice loop	Twenty-two checks driving a fake speech engine, asserting the microphone re-arms and that watchdogs fire.
11	Wake word and proactive voice	Fifty-five checks on policy gates and, when the library is present, the real model against real audio.
12	Hands-free interface wiring	Ten checks that wake and speak events actually reach the interface.
13	Research layer	Sixty-eight checks, one per claim in the paper.
14	Live endpoint sweep	Boots the server and calls every registered route, reporting status, timing and whether any returned a server error.
15	Browser route smoke (optional)	Loads all routes in a real browser and screenshots each.

Table 18 — The fifteen verification gates, run by a single command.

Three principles extracted from building this

Test the claim, not the code. The research suite contains one check per claim in the paper. If a claim cannot be tested, it should not be in the paper. Several of the domain-model defects in Part VI were found by this discipline, because writing the assertion forced the question “what would this look like if it were broken?”

Prefer the failing case to the passing one. The most valuable tests written were the negative controls: silence must not trigger the wake word; unrelated memories must not be reported as duplicates; a forgotten memory must not appear in *any* retrieval mode. A suite composed only of positive cases will pass on a system that does nothing.

Prove the check can fail. During stabilisation, the crash-class bug was deliberately re-introduced and the gate was confirmed to refuse it. This is the only way to know that a green result means something. It is also how three false-positive defects in the checkers themselves were found — each generated by testing the test.

Part VIII — The Research Contribution

The paper's central research question is: *how can a personal AI continuously update, consolidate, prioritise and temporally reason over a user's evolving knowledge without accumulating redundant, outdated or contradictory memories?*

The seven components

#	Component	Function	Key design decision
C1	Adaptive Memory Scoring	Eight-term importance score with every term persisted	Auditable rather than opaque: a score can be explained term by term
C2	Temporal Personal Knowledge Graph	Triples with validity intervals and state supersession	Close facts rather than delete them, so history stays queryable
C3	Contradiction-Aware Retrieval	Three conflict classes detected; rank damping applied	Damp rather than remove, preserving both current and historical answers
C4	Memory Consolidation	Near-duplicate clustering with provenance retention	Threshold calibrated from the measured distribution, not assumed
C5	Knowledge-Gap Detection	Exposure versus depth, weighted by graph connectivity	Connectivity factor prevents flagging well-understood concepts
C6	Retrieval-Enforced Forgetting	Exclusion at five independent surfaces	Enforcement at retrieval, not deletion, because derived stores survive deletion
C7	PersonalBrain-Bench	Synthetic corpus with six question categories	Synthetic by design, so the experiment is reproducible and contains no private data

Table 19 — The seven contributions and the decision that defines each.

Measured results

Pipeline	Hit@5	MRR	Stale@1	Leak	Duplicate	Latency
Vanilla dense	1.000	0.833	0.364	0.022	0.022	39 ms
Hybrid	1.000	0.944	0.182	0.022	0.022	33 ms
Graph-augmented	1.000	0.944	0.182	0.022	0.022	42 ms
Adaptive (proposed)	1.000	1.000	0.000	0.000	0.000	84 ms

Table 20 — Comparative results on PersonalBrain-Bench ($k = 5, 11$ questions, 15 memories, corpus-scoped evaluation). The quality metrics are deterministic: consecutive runs reproduce them exactly, on a database holding the corpus alone or the corpus beside hundreds of unrelated memories. Latency is wall-clock and varies between runs.

Question category	Vanilla MRR	Hybrid MRR	Adaptive MRR	Change
Factual	1.000	1.000	1.000	—
Temporal	0.750	0.750	1.000	+0.250
Contradiction	0.500	1.000	1.000	+0.000
Multi-hop	1.000	1.000	1.000	—
Duplicate	1.000	1.000	1.000	—

Table 21 — MRR by category. Improvements appear exactly where the architecture predicts them and nowhere else.

What the results actually mean

The headline finding is negative, and that is the contribution. Hit@5 is 1.000 for every pipeline including the simplest baseline. Every system retrieves the relevant memory somewhere in its top five. Had the evaluation used only recall, the conclusion would have been that seven research components accomplished nothing measurable, and that conclusion would have been wrong.

What differs is the *ordering* and the *staleness of what comes first*. The proposed pipeline raises mean reciprocal rank from 0.833 to 1.000 and reduces the rate of leading with superseded content from 36.4% to 0.0%. Because a language model consumes the ordered context and leads with what it reads first, staleness at rank one is what produces an answer that is confidently wrong — and, critically, every retrieved passage remains faithful to its source, so no faithfulness metric would flag it.

The category breakdown localises the gains. Factual, multi-hop and duplicate questions are unchanged at 1.000, which is a desirable property: it means the added machinery is not a general-purpose change to retrieval but a targeted one. Temporal questions improve from 0.750 to 1.000 once predicate canonicalisation makes supersession fire. Contradiction questions are where the vanilla baseline is weakest (0.500), consistent with the motivating scenario.

The forgetting result is the sharpest. The baselines do not merely score lower; they *continue to return content the user explicitly asked to remove*. The adaptive pipeline returns it zero times. This is evidence that removal from a retrieval system is a property of the retrieval layer and is not obtained by deleting a row from the primary table.

Limitations, stated plainly

- **The benchmark is synthetic and small.** Fifteen memories and eleven questions across six categories. The categories isolate specific behaviours; they do not represent the distribution of real personal data. Absolute values should not be extrapolated.
- **Extraction is rule-based and conservative.** It will miss phrased contradictions that do not match its patterns. This trades recall for precision deliberately — a wrong fact in a personal graph is worse than a missing one — but the trade is real.
- **Temporal handling uses UTC-normalised timestamps** without full timezone or partial-interval reasoning, which would matter for events spanning midnight across zones.
- **The evaluation covers retrieval, not generation.** Whether higher MRR and lower stale@1 translate into fewer incorrect answers in generated prose is not measured here. This is the natural next experiment and the most significant gap.
- **The ablation is not yet factorised.** The per-category results localise the effect, but an independent run isolating each of the five post-retrieval stages has not been performed. The paper states this rather than inferring it.

The most defensible sentence in the paper

“Every baseline continues to return explicitly unlearned content.” This is a negative result about the baselines, it is measured, it is reproducible, and it is not obvious in advance. A contribution does not have to be a large positive number; a precise demonstration that an apparently simple property is not automatic is equally publishable, provided it is honest about scope.

Part IX — Current State and Future Work

What works today

Capability	Status	Evidence
Automatic screen capture with OCR	Working	Continuous capture with information-density scoring and hero-frame election
Memory vault with GitHub sync	Working	123 knowledge cards, auto-committed every 60 seconds
Self-improving wiki	Working	93 articles across six domains, recompiled as evidence arrives
Knowledge graph	Working	350 nodes, 500 edges, interactive visualisation
Question answering over memory	Working	Local model over 49-table store; verified by the RAG retrieval assertion
Mail ingestion	Working	Read-only IMAP; 47 automated checks; deadlines become tasks and spoken alerts
Calendar ingestion	Working	iCalendar feeds with recurrence expansion and timezone normalisation
Voice in and out	Working	Recogniser re-arms correctly; watchdogs prevent the previously reported stall
Wake word	Working when installed	Measured 0.9968 against 0.000 for silence
Proactive speech	Working	Priority-gated with quiet hours, cooldown and dedupe
Research layer	Working	77 checks; four retrieval pipelines with measured comparison
Desktop packaging	Partial	Runs from source; an installer configuration exists but has not been produced

Table 22 — Current capabilities with the evidence for each.

What does not work, or is not attempted

- **Writing to the user's accounts.** Every integration is read-only by design. The system can draft a reply but cannot send it, and can report a deadline but cannot create the calendar event. This is a deliberate safety choice rather than a limitation of capability, and reversing it would require an approval layer that has not been built.
- **Generative answer quality measurement.** Retrieval is measured; generation is not. Whether the improved ordering produces measurably better prose has not been tested.
- **Long-running autonomy.** Agents run on schedules and timers within the running application. There is no daemon, so nothing happens when the application is closed.
- **Multi-user support.** The schema has a user table and the authentication is real, but the deployment model is single-user and the vault is a single Git repository.
- **Installer.** A configuration exists but a distributable installer has not been built.

Immediate next steps

Priority	Work item	Rationale
1	Factorised ablation	The one gap the paper admits. Each of the five post-retrieval stages isolated, so the contribution of each is measured rather than inferred.
2	Generation-quality study	Establish that the retrieval gains survive into the answer. A blind comparison across the four pipelines on the same question set.

Priority	Work item	Rationale
3	Grow PersonalBrain-Bench	More memories, more categories, adversarial cases designed to break the conflict detector. Synthetic construction keeps it reproducible.
4	Grounded answering with provenance	The retrieval trace already records which memories were used; surfacing citations in the answer makes the reasoning auditable to the user.
5	Approval-gated write actions	Draft the reply, propose the calendar event, and require confirmation. Turns the system from an advisor into an assistant without giving up the safety property.

Table 23 — Prioritised next steps.

Closing assessment

The system does what the original brief asked. It captures continuously, stores durably, answers in plain language, speaks and listens, and runs entirely on the authors' machine without a paid service. It also does something the original brief did not anticipate: it identifies and addresses a class of failure specific to personal memory that general-purpose retrieval evaluation does not measure.

The honest summary of the research contribution is that it is a careful negative result surrounded by a positive one. The negative result is that recall is the wrong instrument for personal memory — it saturates and conceals. The positive result is that three custom metrics — staleness at rank one, leakage after forgetting, and within-query duplication — do discriminate, and that a five-stage post-retrieval layer improves all three to zero while leaving ordinary recall untouched. That is a modest but real and defensible finding.

The project's most transferable output may be neither the application nor the paper but the verification discipline described in Part VII. Of the twenty-seven defects recorded in Part VI, twenty-five were found by an automated check rather than by a person noticing something wrong. The two that a person found — a missing import and a stalled microphone — were both immediately converted into automated checks. The pattern that emerges is that *the value of testing is not in catching bugs that exist; it is in forcing the specification of what correct behaviour would even look like.*

Appendix A — Project Statistics

Metric	Value
Total commits	231
Substantive commits	69
Automated vault synchronisations	162
Python source files	165
Python lines of code	17,149
JavaScript/JSX files (application)	55
JavaScript/JSX lines of code	12,449
Backend route modules	21
Specialised agents	10
Database tables	49
Interface routes	19
Automated verification gates	15
Test suites	8
Research assertions	77
Documented defects	27
Documented deviations	13

Table 24 — Project metrics at the time of writing.

Appendix B — Build Phases at a Glance

PHASE 0	Aug 2026	Foundations: FastAPI, SQLite, auth, capture skeleton
PHASE 1	17 Sep	Curation, hero images, vault, wiki compiler, 3D brain
PHASE 2	18 Sep	Free native scrapers, insight collisions, spoken briefing
PHASE 3	21 Sep	Interface rebuilds: obsidian -> stark -> 15-tab OS + orb
PHASE 4	22 Sep (am)	Two full error audits, verification system created
PHASE 5	22 Sep	Google OAuth, then IMAP + iCal, mail ingestion pipeline
PHASE 6	22 Sep	Wake word, proactive voice, orb voice-loop repair
PHASE 7	22 Sep	Research layer: 7 contributions, benchmark, IEEE paper
Gates added : 0 -> 15		Defects found and fixed : 27
Deviations : 13		Assertions : 8 suites, ~400 individual checks

The build chronology in one view. Note that phases 4 through 7 all occurred on the same day; the pace reflects the leverage gained once the verification system existed.

Appendix C — Glossary

Term	Definition
Agent	A component with a single responsibility, its own failure mode, and no knowledge of the others.
App password	A long random credential issued by a provider for legacy protocols, usable after two-factor authentication is enabled. No consent screen, no expiry.
CASA	Cloud Application Security Assessment. A third-party security audit required by Google for applications publishing restricted scopes.
Chunk	A passage of text small enough to embed meaningfully and store as one unit.

Term	Definition
Consolidation	Merging near-duplicate memories into one canonical record while retaining the identifiers of everything merged.
Cosine similarity	The cosine of the angle between two vectors; the standard measure of semantic similarity.
Damping	Multiplying a retrieved item's ranking weight to demote it without removing it.
Embedding	A numeric vector representing text, positioned so that similar meanings are nearby.
Exposure versus depth	Having encountered a concept versus having understood it. The distinction knowledge-gap detection must make.
Hero image	The single representative frame elected from an information period, kept instead of every screenshot.
Hit@k	Whether a relevant item appeared in the top k results.
IMAP	Internet Message Access Protocol. The mechanism desktop mail clients use to read mailboxes.
iCalendar	A text format for calendar data. Google publishes a private address per calendar requiring no authentication.
Knowledge graph	Entities as nodes and typed relationships as edges.
Leak rate	Fraction of returned results that were explicitly forgotten. Custom metric introduced by this work.
MRR	Mean reciprocal rank: the mean of $1/\text{rank}$ of the first relevant result. Rank-sensitive.
OAuth	A protocol allowing an application limited access to an account without receiving the password.
RAG	Retrieval-augmented generation. Retrieving relevant passages and passing them to a language model along with the question.
Scope	A named permission bundle in OAuth. Providers tier scopes by sensitivity.
Stale@1	Whether the top-ranked result is outdated. Custom metric introduced by this work.
Supersession	Closing an older fact when a newer one replaces it, rather than deleting it.
TF-IDF	A classical term-weighting scheme for keyword retrieval. Needs no neural network, which makes it deterministic.
Unlearning	Removing the influence of specific data from a system that has already incorporated it.
Validity interval	The time range during which an assertion was true. A null end means still true.
Vector store	An index optimised for nearest-neighbour search over embeddings.
Wake word	An always-listening keyword spotter that fires when a phrase such as the assistant's name is recognised.